

# Evolução de Infraestrutura: Do Monólito aos Microserviços

## Estudo de Caso Contextualizado: O Desafio da Startup AppFood

A startup **AppFood** iniciou suas operações validando o mercado com uma arquitetura monolítica convencional. Atualmente, o sistema centraliza quatro domínios de negócio essenciais no mesmo ecossistema de implantação: **Vendas**, **Pagamentos**, **Usuários** e **Pedidos**. Todos compartilham o mesmo poder computacional e utilizam uma única instância de banco de dados relacional.

Com o recente escalonamento no volume de acessos diários, gargalos críticos começaram a surgir. Picos de acesso no módulo de *Pedidos* geram exaustão de conexões no banco de dados, afetando a autenticação de *Usuários*. Adicionalmente, qualquer falha ou exceção não tratada no módulo de *Pagamentos* derruba toda a aplicação, indisponibilizando o serviço e gerando prejuízos financeiros severos. A diretoria de tecnologia determinou a migração imediata para uma arquitetura baseada em **Microserviços Orquestrados**.

## Objetivo Geral da Atividade

Os estudantes deverão atuar como Engenheiros DevOps e Arquitetos de Soluções. O objetivo é projetar o planejamento de transição desse ambiente monolítico para uma topologia moderna de microserviços descentralizados, estabelecendo fluxos automatizados de integração, empacotamento estável, resiliência na orquestração e coleta distribuída de telemetria.

## Etapa 1: Engenharia de Requisitos & Organização Ágil

Antes de estruturar a infraestrutura tecnológica, é imperativo segmentar o escopo do monólito a partir das regras de negócio existentes. Os alunos devem realizar as seguintes tarefas:

### 1. Mapeamento de Domínios e Requisitos

Identificar e documentar rigorosamente **2 requisitos funcionais técnicos** para cada um dos quatro módulos isolados. Estes requisitos guiarão o dimensionamento futuro dos contêineres:

- **Módulo de Usuários:** Escopo voltado à autenticação segura, tokens de acesso e perfis de clientes.
- **Módulo de Pedidos:** Escopo voltado à persistência de carrinhos de compras, fluxos de checkout e regras de transição de estados.
- **Módulo de Pagamentos:** Escopo focado em integrações externas com gateways, estornos e processamento assíncrono.

- **Módulo de Vendas:** Escopo voltado a relatórios analíticos, faturamento consolidado e gerenciamento de cupons ativos.

## 2. Gestão de Fluxo via Kanban Digital

Estruturar as demandas de migração utilizando exclusivamente um quadro **Kanban Digital** (ex: Jira Software, Trello ou GitHub Projects). O fluxo de trabalho deve respeitar estritamente o ciclo de qualidade moderno através das seguintes colunas obrigatórias:

| COLUNA      | DESCRIÇÃO DO ESTADO DO CARD                                                              |
|-------------|------------------------------------------------------------------------------------------|
| To Do       | Backlog técnico priorizado pronto para ser puxado pela equipe de engenharia.             |
| In Progress | Tarefas ativas sob desenvolvimento de código ou escrita de manifestos de infraestrutura. |
| Code Review | Manifestos e códigos concluídos aguardando validação cruzada entre membros do time.      |
| Testing     | Ambiente de Staging onde testes automatizados ou validações de carga são processados.    |
| Done        | Artefatos validados, implantados em produção e monitorados com sucesso.                  |

**Diretriz Técnica para o Kanban:** Os alunos deverão cadastrar obrigatoriamente cards de engenharia estrutural no quadro, tais como: "Escrever Dockerfile multi-stage para Pagamentos", "Configurar segredos de conexão no Kubernetes" e "Desenhar alertas de latência no Prometheus".

## Etapa 2: Automação com Pipeline de CI/CD e Ciclo do Docker

A separação em microsserviços exige que cada componente possua seu repositório de código isolado no **Git** e sua própria esteira automatizada de Continuous Integration (CI) e Continuous Deployment (CD).

### 1. Etapas Obrigatórias da Esteira (Pipeline)

Os alunos devem mapear logicamente e descrever as ações executadas em cada etapa da pipeline de automação:

- **Build** Execução da compilação dos binários correspondentes à stack do serviço, garantindo a integridade sintática primária.
- **Lint & Quality** Análise estática do estilo de escrita de código para assegurar conformidade com guias de estilo oficiais da linguagem.
- **Test** Disparo automatizado de testes unitários e de integração, exigindo cobertura mínima para evitar regressões.

- **SAST** (Static Application Security Testing) Varredura ativa em busca de vulnerabilidades conhecidas em dependências e falhas de segurança no código.
- **Package (Artefato)** Empacotamento do binário/código validado, preparando-o para a etapa subsequente de containerização.

## 2. Fluxo de Empacotamento com Docker

Demonstrar o ciclo de vida do artefato através do ecossistema de contêineres, detalhando:

1. A construção da **Imagem Docker** focando em otimização de tamanho (utilizando imagens base como Alpine ou Distroless).
2. O armazenamento e versionamento da imagem gerada em um **Container Registry** privado e seguro.
3. A instanciação controlada da imagem operando isoladamente como um **Container** dinâmico.

## Etapa 3: Orquestração e Desacoplamento com Kubernetes

A infraestrutura de destino abandonará a máquina virtual única para adotar um ecossistema gerenciado por um cluster **Kubernetes**, visando tolerância a falhas e alta disponibilidade.

### 1. Isolamento Estrutural de Dados: Padrão Database-per-Service

Os estudantes devem fundamentar os motivos de engenharia que justificam o abandono do banco de dados compartilhado. Deve-se analisar o impacto do desacoplamento e como a independência de dados evita gargalos transacionais, permitindo que falhas em um banco não causem indisponibilidade nos demais microsserviços.

### 2. Mapeamento Topológico do Cluster

O desenho operacional da infraestrutura deve detalhar o comportamento dos seguintes componentes:

- **Load Balancer:** Ponto único de entrada de tráfego externo da rede pública, responsável por distribuir as requisições recebidas de forma homogênea.
- **Nodes e Cluster:** A infraestrutura de computação agregada (Nodes físicos ou virtuais) que compõe o cluster, garantindo recursos de hardware para as aplicações.
- **Pods (Vendas, Pagamentos, Usuários e Pedidos):** As unidades mínimas de execução do Kubernetes, operando instâncias individuais de cada microsserviço isolado com réplicas dinâmicas.
- **Frontend:** Alocação segregada do componente de interface com o usuário, impedindo que o consumo do cliente dispute CPU diretamente com os workloads pesados de backend.

## Etapa 4: Observabilidade e Telemetria Distribuída

Em ambientes distribuídos complexos, o rastreamento manual de bugs torna-se inviável. A saúde da infraestrutura deve ser monitorada em tempo real através de duas frentes complementares:

1. **Coleta e Centralização de Logs e Métricas:** Implementação do **Prometheus** atuando ativamente via modelo pull para raspar métricas de performance (consumo de memória, uso de CPU, taxas de erro HTTP e latência) diretamente de cada Pod exposto no cluster.
2. **Visualização Analítica:** Configuração do **Grafana** acoplado ao Prometheus para consolidar painéis visuais interativos (Dashboards), permitindo a rápida identificação de anomalias no microserviço de *Pagamentos* ou de *Vendas* de forma isolada.

### Exemplo Prático de Métrica para Monitoramento:

```
http_requests_total{job="pagamentos-service", status="500"}[5m]
```

Esta expressão permite mensurar picos anômalos de erros internos nas transações de pagamento nos últimos 5 minutos.

## Entregáveis Exigidos dos Alunos

O projeto final consolidado por cada equipe deverá conter rigorosamente os dois componentes técnicos abaixo:

### 1. Desenho Arquitetural da Solução

Apresentação detalhada da nova topologia em microserviços através de um diagrama arquitetural limpo e descritivo. O desenho deve ilustrar claramente o fluxo completo da requisição: a entrada pelo **Load Balancer**, o roteamento inteligente para os **Pods de microserviços** específicos dentro dos respectivos **Nodes**, a comunicação com seus **bancos de dados isolados** e o isolamento do componente de **Frontend**.

### 2. Documento de Fundamentação Técnica

Memorial descritivo detalhando cada tecnologia adotada no ecossistema proposto (Git, Pipeline CI/CD, Docker, Kubernetes, Prometheus e Grafana). Para cada uma delas, o aluno deve redigir uma justificativa clara contendo:

- **O que é** a tecnologia em sua essência técnica.
- **Para que serve e qual o motivo exato** de sua aplicação no cenário crítico enfrentado pela startup AppFood.